

Ask the Airport

Construa uma aplicação de IA generativa em uma tarde. Traga uma ideia, escolha sua stack, publique um repositório público. Quanto mais da plataforma você usar, mais pontos você faz — mas nada impede você de competir.

DATA
qua, 02/09/2026

SPRINT
2h30

TIMES
10 · 4 a 5
pessoas

DEMO
até 2 min

PONTOS
100

LEIA ISTO PRIMEIRO

O evento acontece na **quarta-feira, 2 de setembro de 2026**, em São Paulo. Tudo o que é entregue no dia — conta AWS, chave do Bedrock, instância EC2 e prefixo no S3 — é **por time**, compartilhado entre os integrantes.

Atenção à divisão de regiões: o **console da AWS e a EC2 estão em São Paulo (sa-east-1)**, e o **Bedrock está em Singapura (ap-southeast-1)**. Isso é proposital, não é bug.

O desafio

Uma aplicação de IA. Esse é o briefing — o resto é com você.

Não vamos definir o problema por você. Traga um ponto de vista: um assistente, uma ferramenta de análise, um copiloto, um agente, algo em que não pensamos. A avaliação premia ideias inteligentes e valor de negócio real muito mais do que marcar caixinhas.

Caminhos que valem a pena roubar

- **Análise em linguagem natural** — entra uma pergunta, sai o SQL gerado e a resposta.
- **Copiloto de disrupção** — cruze voos e clima para prever quais conexões vão quebrar, e explique o porquê em linguagem simples.
- **Agente de viagens com memória** — aprenda as preferências de um passageiro em uma conversa e aplique na seguinte. Remarque o voo dele quando atrasar três horas.
- **Concierge semântico** — busca vetorial sobre rotas e anotações para o clássico “me acha algo parecido com isto”.
- **Algo completamente diferente**. Sério — os pontos de criatividade são de verdade.

O dataset airportdb

O campo de jogo sugerido. Você não é obrigado a usá-lo.

Preparamos um banco realista de companhias aéreas: aeroportos, voos, reservas, passageiros e clima, filtrado para os voos que tocam o Brasil em uma semana de junho de 2015. É pequeno o bastante para importar em minutos e rico em problemas interessantes — atrasos que se propagam, conexões que quebram, clima que atrapalha, passageiros com histórico e preferências. Times que acham um problema real nesses dados e o resolvem bem têm vida mais fácil para pontuar em criatividade e valor de negócio do que quem começa da folha em branco.

ITEM	DETALHE
Janela	2015-06-02 a 2015-06-09 (uma semana)
Escopo	Todos os voos que tocam um aeroporto brasileiro, mais uma amostra global para variedade
Volume	5.191 voos · 617.062 reservas · 36.095 passageiros · dados de clima incluídos
Tabelas	12 no total: airline airplane airplane_type airport airport_geo booking employee flight flightschedule passenger passengerdetails weatherdata
Download	hackaton-tidb.s3.sa-east-1.amazonaws.com/dumps/hackathon_airportdb.sql.gz — 10 MB compactado

Como pontuar

São 100 pontos, divididos entre o que você construiu e como você construiu. A Parte A é julgada ao vivo na sua demo. A Parte B é verificada por IA lendo o seu repositório público.

PARA SER AVALIADO, VOCÊ PRECISA DE

Um **repositório público no GitHub**, um **SUBMISSION.md** na raiz dele, **TiDB usado em algum ponto** da aplicação e uma **demo funcionando de no máximo 2 minutos**. Esse é o piso inteiro. Todo o resto vale pontos, não é barreira.

Parte A — o que você construiu

JULGADO AO VIVO, A PARTIR DA SUA DEMO

60 pts

20

Funciona

A coisa roda e faz o que você diz que faz. Um app tosco que funciona ganha de um app lindo que não funciona.

15

Inovação e criatividade

Um ângulo que a gente ainda não viu quatro vezes hoje. Surpreenda a banca.

15

Valor de negócio

Quem tem esse problema, e isso é mesmo uma resposta para ele? Dê nome ao usuário.

10

Demo e pitch

Dois minutos, uma história clara. Mostre a coisa funcionando em vez de descrevê-la.

Parte B — como você construiu

VERIFICADO POR IA NO SEU REPOSITÓRIO PÚBLICO

40 pts

10

TiDB Cloud Starter na AWS

Seus dados vivem em um cluster TiDB Cloud na AWS de São Paulo (sa-east-1). Rodar TiDB local não tira sua elegibilidade — só não pontua aqui.

8

Busca vetorial no TiDB

Uma coluna VECTOR consultada com VEC_COSINE_DISTANCE, ou a função de auto-embedding EMBED_TEXT. As duas contam.

8

Amazon Bedrock

Usado para geração de texto, embeddings ou os dois. Qualquer outra API de LLM mantém você totalmente elegível — você só não leva estes 8 pontos.

8

Publicado na AWS

O alvo de deploy é a instância EC2 do seu time em sa-east-1. Qualquer outro serviço AWS acessível por uma URL também vale. Rodar no seu notebook é aceitável; só vale zero aqui.

6

Construído com Kiro

Commite suas specs e steering files em .kiro/ para vermos o trabalho orientado a spec, não só o resultado.

A CONTA HONESTA

Um time que roda tudo localmente, sem nenhuma conta AWS, ainda pode fazer **60 pontos mais a busca vetorial**.

Um time que usa a plataforma inteira chega a 100. A diferença existe, mas não é um muro — um build local brilhante ganha de um build publicado sem graça.

O que é entregue no dia

Tudo abaixo é por time, não por pessoa. Uma conta, uma chave, uma máquina, um prefixo — compartilhados entre os 4 ou 5 integrantes.

RECURSO	O QUE VOCÊ RECEBE
Conta AWS	Um usuário IAM no formato latam-hackathon-0XX, com login em tidb-latam-hackathon.signin.aws.amazon.com/console. Cadastre o MFA no primeiro acesso — a chave da API do Bedrock só é liberada depois disso.
Chave do Bedrock	Uma chave por time, emitida para ap-southeast-1 (Singapura). Não compartilhe credenciais entre times.
Instância EC2	Uma por time em sa-east-1, com a tag do nome do time. Acesso pelo Session Manager, sem chave SSH. As portas 8000-8999 e 3000 estão abertas para entrada — suba sua aplicação em 0.0.0.0, nunca em 127.0.0.1.
Prefixo no S3	Um bucket compartilhado, um prefixo por time: s3://tidb-latam-hackathon-2026-048364544505/latam-hackathon-0XX/
TiDB Cloud	Não é entregue: cada time cria o próprio cluster Starter gratuito em tidbcloud.com. A organização nunca fica com essas credenciais.

Kiro

Não é entregue. **Instale antes de chegar**, em <https://kiro.dev>, e já faça login. Os créditos saem no dia; a instalação não pode comer o seu sprint.

ANTES DE CHEGAR (VINTE MINUTOS EM CASA)

Conta no GitHub · conta no TiDB Cloud (gratuita, sem cartão) · Python 3.9+ e pip funcionando, ou a linguagem que você preferir · Kiro instalado e logado.

A EC2 do seu time troca de IP público a cada stop/start — não há Elastic IP. Não amarre nada a esse IP e, principalmente, **não restrinja o acesso do TiDB Cloud a ele**: você perderia o banco no meio do sprint. O ajuste padrão de acesso do Starter serve muito bem para um cluster descartável de 2h30.

PRAZO DE VALIDADE

Depois de **05/09/2026**, todas as contas AWS, instâncias EC2, dados no S3, clusters TiDB e chaves de API são apagados. Leve para casa o que importa antes disso — seu repositório público é o que sobra.

Bedrock em Singapura

O console e a EC2 estão em São Paulo. O Bedrock está em Singapura. Isso é esperado.

A chave do Bedrock do seu time foi emitida para `ap-southeast-1`. Se você apontar o cliente para `sa-east-1` — ou para qualquer outra região — vai tomar um erro de autenticação ou de acesso negado, e não um erro dizendo “região errada”. Guarde esse sintoma: **erro de credencial no Bedrock quase sempre é região errada**. Comece copiando o `.env` abaixo, que já vem com a região certa.

.env pronto para copiar

```
# Bedrock – Singapura
AWS_BEARER_TOKEN_BEDROCK=<chave-do-seu-time>
AWS_REGION=ap-southeast-1

# TiDB Cloud Starter – São Paulo
TiDB_HOST=<host>.clusters.tidb-cloud.com
TiDB_PORT=4000
TiDB_USER=<usuario>
TiDB_PASSWORD=<senha>
TiDB_DATABASE=airportdb
```

FINALIDADE

MODEL ID DISPONÍVEL EM `ap-southeast-1`**Texto**`anthropic.claude-3-5-sonnet-20240620-v1:0`**Texto (mais rápido)**`anthropic.claude-3-haiku-20240307-v1:0`**Embeddings**`cohere.embed-english-v3` — 1024 dimensões**Embeddings
multilíngues**`cohere.embed-multilingual-v3` — 1024 dimensões

Cliente Python, com a região já correta

```
import json, os, boto3
from dotenv import load_dotenv

load_dotenv()

bedrock = boto3.client("bedrock-runtime", region_name="ap-southeast-1")

def perguntar(prompt: str) -> str:
    resp = bedrock.invoke_model(
        modelId="anthropic.claude-3-haiku-20240307-v1:0",
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 500,
            "messages": [{"role": "user", "content": prompt}],
        }),
    )
    return json.loads(resp["body"].read())["content"][0]["text"]
```

DOIS DETALHES QUE ECONOMIZAM TEMPO

Titan e Mistral não existem em ap-southeast-1. Se você copiar um `modelId` de um tutorial e ele não estiver na tabela acima, vai receber `ValidationException`.

A latência até Singapura é real: algumas centenas de milissegundos por chamada, ida e volta. Agrupe o trabalho em poucas chamadas grandes; não encadeie o Bedrock dentro de um loop linha a linha.

TiDB Cloud e busca vetorial

Do cluster à primeira consulta em uns dez minutos.

1. Crie o cluster

Em tidbcloud.com, crie um novo recurso. Escolha o plano **Starter** (gratuito), Cloud Provider **AWS** e Region **São Paulo (sa-east-1)**. Ele fica ativo em um ou dois minutos. O plano gratuito dá 5 GiB e 50 milhões de request units por mês, muito mais do que este dataset precisa.

Abra o cluster, clique em **Connect**, mantenha **Public Endpoint** e copie host, porta (4000), usuário e nome do banco. A senha aparece uma única vez — salve agora ou gere outra depois. Nunca commite essas credenciais: o repositório é público.

2. Carregue o dataset

```
curl -O https://hackaton-tidb.s3.sa-east-1.amazonaws.com/dumps/hackathon_airportdb.sql.gz
gunzip hackathon_airportdb.sql.gz

mysql -h <HOST> -P 4000 -u <USUARIO> -p \
  --ssl-mode=VERIFY_IDENTITY --ssl-ca=/etc/ssl/cert.pem \
  airportdb < hackathon_airportdb.sql
```

O endpoint público exige TLS — é para isso que serve o `--ssl-ca`. No wi-fi do evento, prefira o **import from S3** do console do TiDB Cloud: o cluster baixa o arquivo do lado servidor e nem passa pela sua conexão.

3. Conecte pelo Python

```
pip install pymysql boto3 python-dotenv
```

```
import os, pymysql
from dotenv import load_dotenv

load_dotenv()

conn = pymysql.connect(
    host=os.environ["TIDB_HOST"],
    port=4000,
    user=os.environ["TIDB_USER"],
    password=os.environ["TIDB_PASSWORD"],
    database="airportdb",
    ssl={"ca": "/etc/ssl/cert.pem"},
)

with conn.cursor() as cur:
    cur.execute("SELECT COUNT(*) FROM flight")
    print(cur.fetchone()[0], "voos carregados")
```

4. Busca vetorial pelo caminho curto: EMBED_TEXT

O `EMBED_TEXT()` é do próprio TiDB e **não tem nada a ver com o Bedrock**: não precisa de chave, de região nem de SDK. Você insere texto e o TiDB gera os vetores. É o caminho mais barato para os 8 pontos de busca vetorial.

```
CREATE TABLE flight_notes (
  id BIGINT AUTO_RANDOM PRIMARY KEY,
  flight_id INT,
  note TEXT,
  embedding VECTOR(1024) GENERATED ALWAYS AS (
    EMBED_TEXT("tidbcloud_free/amazon/titan-embed-text-v2", note)
  ) STORED
);

INSERT INTO flight_notes (flight_id, note)
VALUES (1, 'voo direto de Sao Paulo para Frankfurt');

SELECT flight_id, note
FROM flight_notes
ORDER BY VEC_COSINE_DISTANCE(embedding, EMBED_TEXT(
  "tidbcloud_free/amazon/titan-embed-text-v2", 'sem escalas para a Alemanha'))
LIMIT 5;
```

Se preferir gerar os embeddings você mesmo, use `cohere.embed-multilingual-v3` no Bedrock — também são 1024 dimensões, então a definição da coluna `VECTOR(1024)` continua a mesma.

A linha do tempo de 2h30

Um formato que funciona. Ajuste como quiser, mas proteja os últimos quinze minutos — os times que pulam essa parte não submetem.

0:00 — 0:15 Escolham a ideia e dividam o trabalho

Decidam a única coisa que a demo vai mostrar. Distribuam: dados, IA, interface, pitch.

0:15 — 0:30 Cluster de pé, dados dentro

Uma pessoa só. O resto do time já começa a construir com dados falsos.

0:30 — 1:45 Construam o loop principal

Da entrada até a resposta, ponta a ponta, feio mesmo. Faça funcionar antes de fazer bonito.

1:45 – 2:05

Peguem os pontos de stack

Coluna vetorial, deploy na EC2, commit das specs em `.kiro/`. Pontos baratos, vitórias rápidas.

2:05 – 2:20

Escrevam o `SUBMISSION.md` e publiquem

Inegociável. Confiram se o repositório está mesmo público.

2:20 – 2:30

Gravem a demo e ensaiem a fala

Dois minutos. Mostre, não narre.

COMO UM TIME DE CINCO NÃO TRAVA

Um notebook dirige, o time inteiro conduz. Uma pessoa cuida do cluster e do carregamento dos dados enquanto os outros trabalham contra dados falsos — cinco pessoas esperando o mesmo `mysql` < terminar é meia hora jogada fora.

Como submeter

Repositório público no GitHub, mais um comentário na issue fixada “Entregas”. Só isso.

A entrega, em quatro passos

1. Construam no **repositório público do próprio time** no GitHub. Não forkem o repositório principal e não abram Pull Request lá.
2. O repositório precisa conter o código, os specs em `.kiro/` se vocês usaram Kiro, e um `SUBMISSION.md` na raiz: o que construíram, como rodar, o que fariam a seguir.
3. Ao terminar, **comentem na issue fixada “Entregas”** do repositório principal com três coisas: nome do time, URL do repositório e link da demo — vídeo de 2 minutos ou URL no ar.
4. Pronto. A entrega é isso.

O congelamento

No horário do fechamento, a organização **forka todos os repositórios submetidos**. É esse fork que vai a julgamento — e fork não sincroniza. Qualquer commit enviado depois do fechamento simplesmente não é avaliado.

DOIS MINUTOS ANTES DE COMENTAR

Suba tudo primeiro, comente depois. O que não estiver no `main` na hora do fechamento não existe para a banca.

Confirme que o repositório está mesmo público. Repositório privado não pode ser forkado e, portanto, não pode ser pontuado — é a forma mais boba de perder 100 pontos.

Ao submeter, o time concorda que a organização mantenha uma cópia arquivada do repositório — o fork.

O arquivo `SUBMISSION.md`

Crie o `SUBMISSION.md` na raiz do repositório e preencha. Copie este template exatamente — o avaliador de IA lê ele primeiro e depois confere as suas afirmações contra o código de verdade.

```
# SUBMISSION.md

## Time
Nome do time:
Integrantes:

## Pitch
Uma linha sobre o que isto faz e para quem.

## O que faz
Duas ou três frases. Qual problema, qual abordagem, o que o usuário vê.

## Stack – marque o que você realmente usou
- [ ] TiDB Cloud Starter na AWS sa-east-1
- [ ] Busca vetorial no TiDB (coluna VECTOR ou EMBED_TEXT)
- [ ] Amazon Bedrock (ap-southeast-1)
- [ ] Publicado na AWS -> URL no ar:
- [ ] Construído com Kiro (.kiro/ commitado)

## Onde olhar
Conexão/consultas TiDB: caminho/para/arquivo.py
Busca vetorial: caminho/para/arquivo.py
Chamadas ao Bedrock: caminho/para/arquivo.py

## Demo
Link do vídeo de 2 minutos ou da aplicação no ar:
```

UMA REGRA SOBRE AS CAIXINHAS

As afirmações são **verificadas contra o seu código**, não aceitas na confiança. Caixinha marcada que o avaliador não encontra no repositório vale zero naquela linha. Apontar o arquivo certo em “Onde olhar” é o jeito mais rápido de receber crédito pelo que você de fato fez.

Coloque seus segredos no .env e o .env no .gitignore — o repositório é público. Commite um .env.example no lugar.

O que fazer quando algo quebra

Toda falha desta lista tem um desvio que mantém você na competição.

SINTOMA

CORREÇÃO, OU COMO DESVIAR

AccessDeniedException no Bedrock

Confira a região antes de qualquer outra coisa: a chave vale para ap-southeast-1. Se a região estiver certa, falta o formulário de primeiro uso dos modelos Anthropic para a conta — abra qualquer modelo Anthropic no console do Bedrock e preencha. Se travar mesmo assim, troque por outra API de LLM e siga em frente: custa 8 pontos, e mais nada.

ValidationException: modelo não suportado

Esse modelo não existe em ap-southeast-1. Use Claude 3.5 Sonnet, Claude 3 Haiku ou os embeddings da Cohere. Titan e Mistral não estão lá.

Chamada ao Bedrock lenta demais

É a distância até Singapura. Junte vários itens em um prompt só em vez de chamar o modelo dentro de um loop.

Erro de SSL ao conectar no TiDB

O endpoint público exige TLS. No Python, ssl={"ca": "/etc/ssl/cert.pem"}; no CLI, --ssl-ca.

Conexão com o TiDB expira

Veja o Network Access do cluster. O padrão do Starter já serve — se alguém restringiu por IP, desfaça: o IP público da EC2 muda a cada stop/start e derruba o acesso no meio do sprint.

SINTOMA	CORREÇÃO, OU COMO DESVIAR
A aplicação na EC2 não abre no navegador	Você provavelmente subiu em 127.0.0.1. Suba em 0.0.0.0 e use uma porta entre 8000-8999 ou a 3000 — são as únicas abertas. Confirme também o IP público atual, que muda a cada restart.
Import do dataset arrastando	Wi-fi do evento. Use o import-from-S3 no console do TiDB Cloud: o cluster busca o arquivo direto.
Sem acesso à conta AWS	Construa local com qualquer API de LLM. Você mantém os 60 pontos da Parte A e ainda pontua a busca vetorial pelo EMBED_TEXT.
Sem tempo para publicar	Então não publique. Rode local, grave a demo, submeta. Oito pontos não valem uma entrega perdida.

SE VOCÊ TRAVOU DE VERDADE

Chame um mentor antes de queimar quinze minutos sozinho. Ninguém ganha ponto por depurar em silêncio, e sempre existe um desvio que mantém você pontuando.